

1.1

① n 被 p 均匀整除:

每个核心基础元素数量 $q = \frac{n}{p}$

$$\text{my-first-i} = \text{my-rank} \times q$$

$$\text{my-last-i} = (\text{my-rank} + 1) \times q$$

(不包括)

② n 不能被 p 整除

$$q = n / p$$

$$r = n \% p$$

if (my-rank < r) {

$$\text{local-n} = q + 1$$

$$\text{my-first-i} = \text{my-rank} \times (q + 1)$$

}

else {

$$\text{my-first-i} = \text{my-rank} \times q + r$$

} local-n = q

$$\text{my-last-i} = \text{my-first-i} + \text{local-n}$$

1.1 为全局和的例子中计算 `my_first_i` 和 `my_last_i` 的函数设计公式。请记住，在循环中，每个核心应该被分配大致相同数量的计算元素。提示：首先考虑 n 被 p 均匀整除的情况。

答：

1.2 我们隐含地假设每次调用 `Compute_next_value` 需要的工作量与其他调用大致相同。如果调用 $i=k$ 需要的工作量是调用 $i=0$ 时的 $k+1$ 倍，你会如何改变对上一道问题的答案？如果第一次调用 ($i=0$) 需要 2ms，第二次调用 ($i=1$) 需要 4ms，第三次调用 ($i=2$) 需要 6ms，以此类推，你会如何改变答案？

答：

采用循环切片：0号核心拿任务 $0, p, 2p, \dots$ ；1号核心拿任务 $1, p+1, 2p+1, \dots$
使整体耗时平均

答案调整：不再需要计算边界，只要修改 for 循环步长

```
for (int i = my_rank; i < n; i += p) {
```

```
    x = Compute_next_value(i);
```

```
}
```


1.3 试着为图 1.1 所示的树状结构全局和写出伪代码。假设核心的数量是 2 的幂 (1, 2, 4, 8, ...)。提示：使用一个变量 `divisor` 来决定一个核心是否应该发送它的总和或接收并累加。`divisor` 应该从 2 开始，并在每次迭代后增加一倍。还可以使用变量 `core_difference` 来决定哪个核心应该与当前核心配对。它应该从值 1 开始，并且在每次迭代后也要翻倍。例如，在第一次迭代中， $0 \% \text{divisor} = 0$ ， $1 \% \text{divisor} = 1$ ，所以 0 接收和累加，而 1 发送。同样在第一次迭代中， $0 + \text{core_difference} = 1$ ， $1 - \text{core_difference} = 0$ ，所以 0 和 1 在第一次迭代中是配对的。

答：

```

divisor = 2
core_difference = 1
sum = local-value
while (core_difference < p) {
    if (my-rank % divisor == 0) {
        partner = my-rank + core_difference
        Receive (receive-value, partner)
        my-sum = my-sum + receive-value
    }
    else if (my-rank % divisor == core_difference) {
        partner = my-rank - core_difference
        Send (my-sum, partner)
        break
    }
    divisor = divisor * 2
    core_difference = core_difference * 2
}

```

3.1 如果我们使用 `strlen(greeting)` 来表示进程 1, 2, `comm_sz-1` 所发送的消息的长度，而不是 `strlen(greeting)+1`，那么在“greeting”程序中会发生什么？如果我们使用 `MAX_STRING` 而不是 `strlen(greeting)+1` 又会发生什么？你能解释一下这些结果吗？

答：

1. 不加 1：会丢失字符串的结束标志 '\0'，程序表面上输出正常，可能因为系统默认初始数组为 0，但这属于逻辑错误，可能导致打印越界，输出乱码
2. `MAX_STRING`：结果正确，接收方能靠 '\0' 正常截断，但把有效数据和垃圾内存一起发送，极大浪费网络通信带宽，降低性能

1.4 作为前面问题中概述的方法的替代方案，我们可以使用 C 的位操作符来实现树状结构的全局和。为了了解这一点，我们可以写下每个核心编号的二进制（基数为 2）表示，并注意每个阶段的配对情况。

核心	阶段 1	阶段 2	阶段 3
$0_{10}=000_2$	$1_{10}=001_2$	$2_{10}=010_2$	$4_{10}=100_2$
$1_{10}=001_2$	$0_{10}=000_2$	$\times$	$\times$
$2_{10}=010_2$	$3_{10}=011_2$	$0_{10}=000_2$	$\times$
$3_{10}=011_2$	$2_{10}=010_2$	$\times$	$\times$
$4_{10}=100_2$	$5_{10}=101_2$	$6_{10}=110_2$	$0_{10}=000_2$
$5_{10}=101_2$	$4_{10}=100_2$	$\times$	$\times$
$6_{10}=110_2$	$7_{10}=111_2$	$4_{10}=100_2$	$\times$
$7_{10}=111_2$	$6_{10}=110_2$	$\times$	$\times$

从表中我们可以看出，在第一阶段，每个核心与编号最右边第一位不同的核心配对。在第二阶段，继续将核心与编号第二位不同的核心配对；而在第三阶段，核心与编号第三位不同的核心配对。因此，如果我们有一个二进制值的位蒙板 bitmask，第一阶段是 001_2 ，第二阶段是 010_2 ，第三阶段是 100_2 ，我们可以通过“反转”编号中与位蒙板 bitmask 中非零位的位置相对应的位，得到配对的核心编号。这可以通过使用按位异或运算符“^”来完成。

在伪代码中使用顺时针按位异或运算符和左移操作符来实现这个算法。

答：

`bitmask = 1`

`divisor = 2`

`my-sum = local-value`

`while ( bitmask < size ) {`

`partner = bitmask ^ my-rank ;`

`if ( my-rank % divisor == 0 ) {`

`Receive ( receive-val , partner );`

`my-sum += receive-val`

`}`

`else {`

`Send ( my-sum , partner );`

`}`

`bitmask <<= 1 ;`

`divisor *= 2 ;`

`}`